

Algorithmic Analysis of Stochastic Gradient Descent for Malaria Cell Classification Using a Convolutional Neural Network

Felix Rolando Gonzalez Zapatanga¹[0009000017727407]

Yachay Tech University, San Miguel de Urcuquí, Ecuador Analysis of Algorithms
`felix.gonzalez@yachaytech.edu.ec`
<https://yachaytech.edu.ec/>

Abstract. This paper presents an algorithmic analysis of Stochastic Gradient Descent (SGD) applied to the training of a convolutional neural network for binary malaria cell classification. The classification task consists of distinguishing between parasitized and uninfected cell images. Instead of focusing on increasingly complex neural architectures, the project focuses on SGD as the central algorithm: its formal definition, design paradigm, update rule, computational complexity, convergence behavior, and empirical performance. A small convolutional neural network was trained using pure mini-batch SGD, without Adam, transfer learning, or ensemble methods. Three experiments were conducted by varying the learning rate and batch size. The best test accuracy obtained in the current experiment was 71.96%, a precision of 65.15% a recall of 86.00% and an F1-score of 74.14% fo the parasitized class. However, the experiment is useful from an Analysis of Algorithms perspective because it demonstrates the computational trade-offs of SGD: a low-cost iterative update, strong sensitivity to hyperparameters, and gradual convergence across epochs. The report concludes that SGD is simple and computationally efficient per update, but its practical success depends heavily on learning rate selection, batch size, number of epochs, preprocessing, and model capacity.

Keywords: Stochastic Gradient Descent · Algorithm Analysis · Convolutional Neural Network · Malaria Cell Classification · Empirical Complexity

1 Introduction

1.1 SGD

Stochastic Gradient Descent (SGD) is one of the most important optimization algorithms in machine learning and deep learning. It is widely used to train models whose objective functions depend on large data sets, because it updates the model parameters using a single training example or a mini-batch instead of computing the full gradient over the entire data set at every step. This makes

SGD attractive when the number of training samples is large or when exact full-batch optimization becomes computationally expensive.

The purpose of this project is to analyze SGD as an algorithm rather than only to build a high-accuracy classifier. The selected application is binary malaria cell classification. Given a cell image, the model predicts whether the cell is parasitized or uninfected. This application is appropriate for the project because it connects a practical classification problem with a non-trivial iterative optimization algorithm.

In the current implementation, a small convolutional neural network (CNN) is trained using the SGD optimizer. Other optimizers such as Adam, transfer learning methods such as VGG16, and ensemble approaches were intentionally removed so that the algorithmic behavior of SGD could be isolated and analyzed. The analysis includes formal definition, implementation details, design paradigm, best-case, average-case, and worst-case complexity, and empirical behavior under different hyperparameter configurations.

2 Problem Definition and Motivation

The computational problem addressed in this project is binary image classification. The input is a color image of a segmented cell, and the output is a binary label:

$$y = \begin{cases} 1, & \text{if the cell is parasitized,} \\ 0, & \text{if the cell is uninfected.} \end{cases} \quad (1)$$

Let $x_i \in \mathbb{R}^{H \times W \times C}$ denote an input image and $y_i \in \{0, 1\}$ its corresponding label. For this implementation, images are resized to $64 \times 64 \times 3$. The goal is to learn a function

$$f_{\theta}(x_i) \approx y_i, \quad (2)$$

where θ represents all trainable parameters of the neural network. The output of the model is a probability $\hat{y}_i \in [0, 1]$. A threshold of 0.5 is used for final classification.

The problem is important because image-based malaria screening is an example of a real-world classification task where algorithmic efficiency and prediction behavior both matter. However, this report should not be interpreted as a medical diagnostic system. The experimental results are intended for algorithm analysis and educational purposes.

3 Dataset and Preprocessing

The data set used in this project follows the common malaria cell-image structure with two folders: **Parasitized** and **Uninfected**. Each image path and label are stored in a data frame. The class encoding used in the implementation is:

- `Parasitized` = 1
- `Uninfected` = 0

The preprocessing pipeline is intentionally simple. Each image is read using OpenCV, converted from BGR to RGB, resized to 64×64 , converted to floating point format, and normalized to the interval $[0, 1]$ by dividing all pixel values by 255.

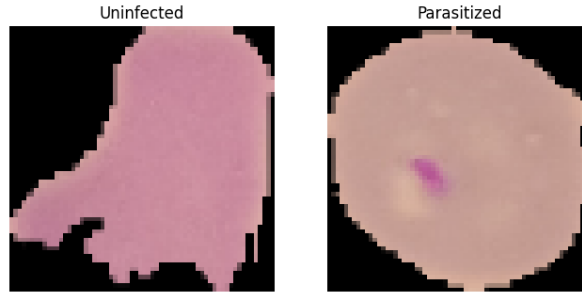


Fig. 1. Examples of uninfected and parasitized cell images after preprocessing.

The data was split using an 80% training and 20% test partition with stratification. During training, 10% of the training set was used as validation data. This allows the report to compare training behavior and validation behavior across epochs.

Here is what we do to get our images ready. The steps we take to prepare our images are as follows:

1. We load the paths to our image files and the labels that go with them.
2. We give a label of 0 to the cells that are not infected and a label of 1 to the cells that are parasitized.
3. We make each image the same size, which is 64 by 64 by 3.
4. We make sure the values of the pixels in our images are, between 0 and 1.
5. We divide our images into two groups: the training images and the testing images.

4 Formal Definition of Stochastic Gradient Descent

Let the training set be

$$D = \{(x_i, y_i)\}_{i=1}^N. \quad (3)$$

The empirical risk minimization problem is defined as

$$\min_{\theta} J(\theta) = \frac{1}{N} \sum_{i=1}^N L(f_{\theta}(x_i), y_i), \quad (4)$$

where L is the loss function and θ contains the trainable weights and biases of the neural network. For binary classification, this project uses binary cross-entropy:

$$L(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]. \quad (5)$$

In full-batch gradient descent, the update would require the gradient over the entire data set. In mini-batch SGD, a mini-batch $B_t \subset D$ is selected at iteration t , and the update is computed using only that mini-batch:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t; B_t), \quad (6)$$

where η is the learning rate. In this implementation, pure SGD is used with `momentum=0.0` and `nesterov=False`. This decision makes the analysis easier because the update rule does not include additional velocity or acceleration terms.

Table 1. CNN architecture used in the SGD experiments.

Layer	Configuration	Purpose
Input	$64 \times 64 \times 3$	RGB cell image
Conv2D	16 filters, 3×3 , ReLU	Local feature extraction
MaxPooling2D	2×2	Spatial reduction
Conv2D	32 filters, 3×3 , ReLU	Deeper feature extraction
MaxPooling2D	2×2	Spatial reduction
Flatten	–	Vector representation
Dense	64 neurons, ReLU	Classification features
Dense	1 neuron, sigmoid	Binary probability

5 Algorithm Description

The following pseudocode describes mini-batch SGD for training the convolutional neural network used in this project.

The Mini-Batch Stochastic Gradient Descent algorithm is used for Convolutional Neural Network training

Input: Training dataset D , learning rate η , batch size B , number of epochs E

Output: Trained model parameters θ

1. Start by initializing the model parameters for the Convolutional Neural Network
 2. For each epoch do the following CNN $e = 1, 2, \dots, E$:
 - 2.1 Shuffle the training dataset to ensure the CNN.
 - 2.2 Split the training dataset into groups called mini-batches of the specified size CNN B .
 - 2.3 For each mini-batch B_t :
 - a. First the CNN makes predictions based on the input data.
 - b. Then calculate the error of the CNN, which is the difference between the predicted output and the actual output.
 - c. Next use backpropagation to determine how much each parameter of the CNN contributed to the error
 - d. Finally update the parameters of the CNN using the learning rate and the gradients $\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t; B_t)$.
 3. After training the CNN for the number of epochs return the trained model parameters of the CNN θ .
-

6 Implementation Details

The implementation was developed in Python using TensorFlow/Keras. The central algorithm is Stochastic Gradient Descent, which was used as the only optimizer during training. Other optimizers such as Adam were intentionally removed because the objective of this project is to analyze SGD directly.

The convolutional neural network receives images of size $64 \times 64 \times 3$. The architecture is intentionally small in order to keep the computational analysis clear and avoid making the project mainly about deep architecture design. The model contains two convolutional layers, two max-pooling layers, one flattening layer, one hidden dense layer, and one output neuron with sigmoid activation.

The architecture is summarized as follows:

The model was trained using binary cross-entropy loss and accuracy as a monitoring metric. Three SGD configurations were evaluated:

- SGD with learning rate 0.01 and batch size 32.
- SGD with learning rate 0.001 and batch size 32.
- SGD with learning rate 0.01 and batch size 128.

We did lots of experiments. Each experiment was trained for 30 epochs. We measured how long it took to train each epoch. Then we looked at the model and checked how well it did using accuracy, precision, recall and F1-score to see how good the final model really was. We used accuracy and precision and recall and F1-score to evaluate the model.

Table 2. CNN architecture trained with SGD.

Layer	Output shape	Parameters
Conv2D, 16 filters, 3×3	$62 \times 62 \times 16$	448
MaxPooling2D	$31 \times 31 \times 16$	0
Conv2D, 32 filters, 3×3	$29 \times 29 \times 32$	4,640
MaxPooling2D	$14 \times 14 \times 32$	0
Flatten	6272	0
Dense, 64 neurons	64	401,472
Dense, 1 neuron	1	65
Total	–	406,625

7 Mathematical Analysis

Let N be the number of training images E the number of epochs and B the mini- size.

The number of parameter updates, per epoch is $\frac{N}{B}$.

$$U = \left\lceil \frac{N}{B} \right\rceil.$$

For each mini-batch, the CNN performs forward propagation, computes the loss, performs backpropagation, and updates the model parameters. Let C_f be the cost of forward propagation for one image and C_b be the cost of backpropagation for one image. The cost of processing one image is approximately:

$$C = C_f + C_b.$$

Therefore, the cost of one epoch is:

$$T_{\text{epoch}} = O(N(C_f + C_b)).$$

For E epochs, the total training complexity is:

$$T_{\text{total}} = O(E \cdot N \cdot (C_f + C_b)).$$

Since the CNN architecture is fixed in this project, C_f and C_b can be treated as constants with respect to the number of images. Under this fixed-architecture assumption, the training complexity becomes:

$$T_{\text{total}} = O(E \cdot N).$$

This simplification does not show the cost of the convolutional layers and the dense layers. The cost of a layer depends on a few things. It depends on the size of the input, the size of the kernel, the number of input channels, and the number of output filters. A convolutional layer has a cost that is approximately.

- The number of input channels.
- The number of output filters.

- The size of the kernel.

The size of the input. The cost of layers and dense layers is important to consider when we talk about convolutional layers. Convolutional layers are used in places and the cost of convolutional layers can be high.

$$O(H \cdot W \cdot K^2 \cdot C_{\text{in}} \cdot C_{\text{out}}),$$

The height and width of the feature map are given by H and W . The kernel size is K . We also have the number of input channels which's C_{in} and the number of filters which is C_{out} . So we are talking about the feature map, with its height and width and the kernel size and the number of input channels and the number of filters.

The dense layer cost is:

$$O(n_{\text{in}} \cdot n_{\text{out}}),$$

where n_{in} is the number of input units and n_{out} is the number of output units.

In this implementation, the dense layer from 6272 inputs to 64 neurons contains most of the parameters. This explains why the model has 406,625 trainable parameters even though the convolutional part is small.

8 Best, Average, and Worst-Case Complexity

For a fixed neural network architecture, the asymptotic complexity of SGD depends mainly on the number of epochs, the number of training samples, and the cost of processing each sample.

8.1 Best Case

The best case occurs when the model reaches an acceptable validation performance in a small number of epochs. If early stopping is used and the model converges after E_{min} epochs, the complexity is:

$$O(E_{\text{min}} \cdot N \cdot (C_f + C_b)).$$

This case is possible when the learning rate is appropriate and the initial parameters allow fast convergence.

8.2 Average Case

The average case corresponds to the expected number of epochs needed to obtain stable convergence under normal training conditions. If the model requires E_{avg} epochs, the complexity is:

$$O(E_{\text{avg}} \cdot N \cdot (C_f + C_b)).$$

In this project, the average practical case was evaluated using 30 epochs for each experiment.

8.3 Worst Case

The worst case occurs when the algorithm requires the maximum allowed number of epochs without reaching a good solution, or when the learning rate causes slow convergence or unstable optimization. If the maximum number of epochs is $E_{\max}$, the complexity is:

$$O(E_{\max} \cdot N \cdot (C_f + C_b)).$$

In the experiments, SGD with learning rate 0.001 showed slower learning and lower recall compared with learning rate 0.01. This indicates that a learning rate that is too small can increase the number of epochs required for good performance.

8.4 Effect of Batch Size

The batch size affects the number of updates per epoch. For the training set of 853 images:

$$U_{B=32} = \left\lceil \frac{853}{32} \right\rceil = 27,$$

$$U_{B=128} = \left\lceil \frac{853}{128} \right\rceil = 7.$$

A bigger batch size means updates per epoch.. Each update handles more images.

In reality, bigger batches can be faster per epoch. However, they might result in fewer parameter updates.

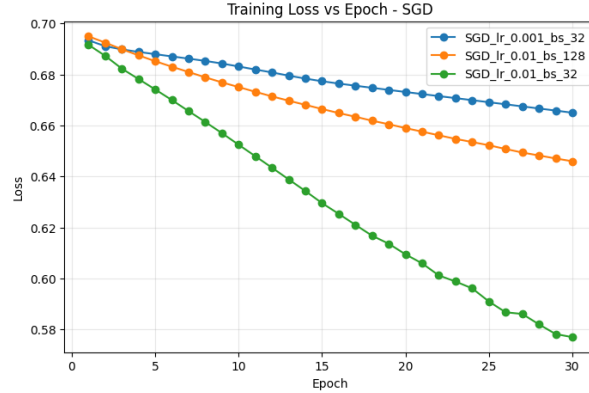
This can lead to improvement in generalization. We saw this happen in our experiments. A batch size of 128 took training time than a batch size of 32. It gave lower recall and F1-score. The batch size 32 produced recall and F1-score. We also noticed that batch size 128 had training time. Overall it did not perform better, than batch size 32.

9 Empirical Analysis

We are looking at three ways to set up SGD. We want to see how the learning rate and the batch size affect how well the model converges, how well it classifies things and how long it takes to run. We did all the experiments the way. They all used the CNN architecture and the same way to split the data into training and testing sets. Each experiment was trained for 30 epochs. We used the SGD configurations for all the experiments to see how the learning rate and batch size of the SGD configurations affect the results.

Table 3. Experimental results after 30 epochs.

Experiment	Acc.	Prec.	Recall	F1	Time (s)	Avg. epoch (s)
SGD lr=0.01, bs=32	0.7196	0.6515	0.8600	0.7414	78.63	2.60
SGD lr=0.01, bs=128	0.6028	0.7143	0.2500	0.3704	73.52	2.44
SGD lr=0.001, bs=32	0.5841	0.7200	0.1800	0.2880	81.68	2.70

**Fig. 2.** Training loss across epochs for the SGD configurations

The best configuration was SGD with learning rate 0.01 and batch size 32. This experiment achieved 71.96% test accuracy and an F1-score of 74.14%. The most important result is the recall of 86.00% for the parasitized class. In this application, recall is especially relevant because false negatives correspond to infected cells classified as uninfected.

The experiment with learning rate 0.001 obtained lower accuracy and lower recall. This suggests that the learning rate was too small for the number of epochs used, causing slow convergence. The experiment with batch size 128 had a slightly lower average epoch time, but its recall was only 25.00%. This indicates that reducing the number of parameter updates per epoch can negatively affect learning in this configuration.

The classification report for the best experiment is shown in Table 4.

Table 4. Classification report for the best SGD configuration.

Class	Precision	Recall	F1-score	Support
Uninfected	0.83	0.60	0.69	114
Parasitized	0.65	0.86	0.74	100
Accuracy	–	–	0.72	214
Macro avg	0.74	0.73	0.72	214
Weighted avg	0.75	0.72	0.72	214

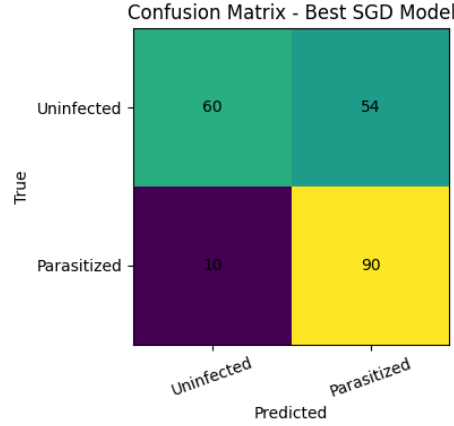


Fig. 3. Confusion matrix for the best SGD configuration in the current run.

10 Convergence Analysis

The convergence curves show that the best configuration, using learning rate 0.01 and batch size 32, improved gradually across the 30 epochs. During the first epochs, the model had performance close to random classification. However, after approximately 15 epochs, the validation accuracy started to increase more clearly. This confirms that SGD may require several epochs before producing stable improvements.

The training and validation loss curves also show that the model continued learning throughout the training process. Compared with the configuration using learning rate 0.001, the learning rate of 0.01 produced faster improvement. This supports the idea that SGD is sensitive to the learning rate.

The batch size comparison also provides an important algorithmic observation. With batch size 128, the number of parameter updates per epoch was smaller. Although this reduced the average epoch time slightly, the final recall and F1-score were much lower than those obtained with batch size 32. Therefore, the best practical configuration was not the one with the lowest runtime per epoch, but the one with the best trade-off between update frequency and convergence quality.

The empirical results are consistent with the theoretical behavior of SGD. Each update has a relatively low computational cost, but the final performance depends strongly on the selected learning rate, batch size, and number of epochs.

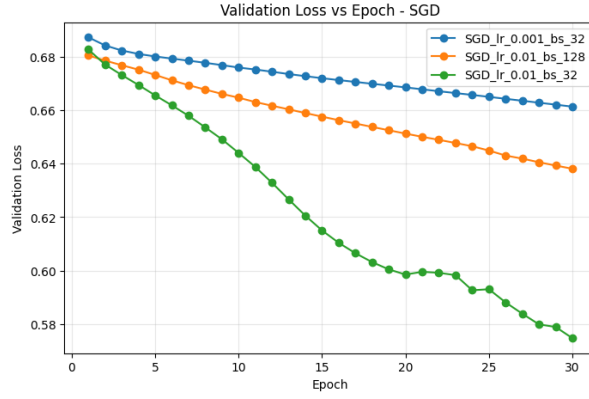


Fig. 4. Validation loss across epochs for the SGD Configurations.

11 Prediction of a New Image

We trained the model using a method called SGD. Then a new picture that the model had not seen before was used to see how it would do. This new picture was treated the way as the pictures used for training. The new picture was. Changed to RGB format. It was also made smaller to 64 by 64. The values in the picture were adjusted to be, between 0 and 1. Then the trained CNN model looked at this picture.

The model produced the following probability:

$$P(\text{Parasitized}) = 0.3940.$$

Since this probability is lower than the threshold of 0.5, the final prediction was:

Uninfected.

This experiment is useful as a qualitative demonstration of the trained model. However, it should not be interpreted as a medical diagnosis. A single external image may have different lighting conditions, resolution, microscope characteristics, or visual distribution compared with the training dataset. Therefore, the main scientific evaluation should be based on the test set metrics rather than on a single external image.

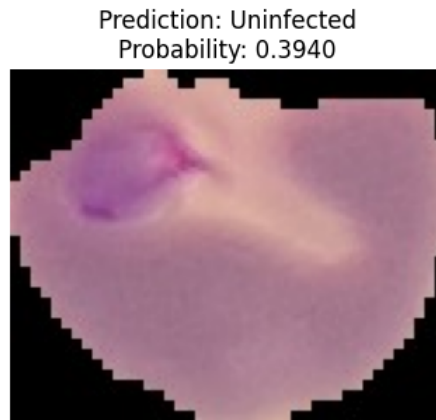


Fig. 5. Prediction for a new external image. The model output probability was 0.3940 and the predicted was Uninfected.

12 Discussion

The results show that SGD works well for training a neural network to classify malaria cells. Its performance really depends on how you schedule it. Best results came from using a learning rate of 0.01 and a batch size of 32.

The test accuracy was 71.96%. Recall for parasitized cells was 86.00%. The SGD method was able to train the network. We have to choose the right training configuration for SGD to work well. SGD is well, for malaria cell classification.

The recall result really matters in malaria detection. If we miss cells it's a bigger problem than wrongly marking healthy cells. The model did a job in finding most infected cells even though it wasn't perfect in identifying healthy ones. It got 65.15% precision, for cells but the recall is high which means it caught most of the infected cells in the test.

The comparison between learning rates shows that a smaller learning rate does not always make the model perform better.

For example the configuration, with a learning rate of 0.001 had accuracy and much lower recall. This suggests that it converges slowly.

Theoretical limitations of Stochastic Gradient Descent or SGD support this. SGD is an algorithm but it needs careful tuning.

The batch size experiment shows us another thing to think about. If we use a larger batch size the computer does updates per epoch and it takes a little less time for each epoch but the classification results are not as good. So we cannot just look at how it takes to run the experiment to decide if the Stochastic Gradient Descent is good or not. We need to think about how much it costs to run the computer and how good the model's at the same time. The batch size experiment is really about finding a balance, between these two things the cost and the model quality of the Stochastic Gradient Descent.

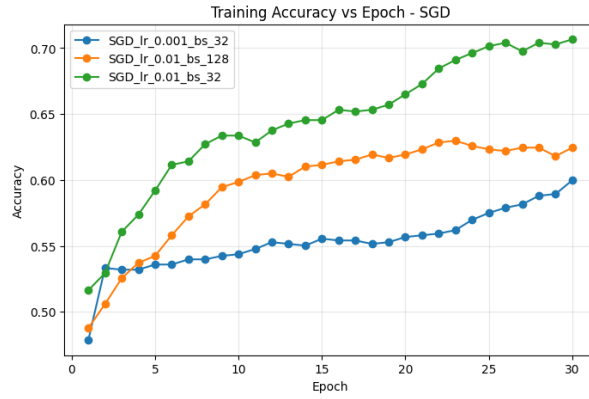


Fig. 6. Training accuracy across epochs for the SGD configurations

13 Limitations

This project has some limitations. First, the experiments were performed using a reduced dataset of 1067 images. A larger dataset would provide a more reliable estimate of the model's generalization performance. Second, the CNN architecture was intentionally simple in order to keep the focus on SGD. More advanced architectures could improve accuracy, but they would also make the algorithmic analysis more complex.

Third they only looked at three SGD configurations. To get an understanding it would be good to try more things like different learning rates, batch sizes, momentum, weight decay and early stopping. But the things they did try are enough to see how SGD works: how it is affected by the learning rate how it is affected by the batch size, how it gets better over time and how long it takes to run. They can use SGD to see these things.

14 Conclusions

This project presented an algorithmic analysis of Stochastic Gradient Descent applied to the training of a convolutional neural network for malaria cell classification. The formal analysis described the SGD update rule, its iterative optimization paradigm, and its computational complexity in terms of epochs, dataset size, batch size, and the cost of forward and backward propagation.

The study found that Stochastic Gradient Descent is a simple way to do things but it is very sensitive to the settings we choose. The best test we did used a learning rate of 0.01 and a batch size of 32 and it got good results, for the parasitized class: it was right 71.96 percent of the time precise 65.15 percent of the time remembered 86.00 percent of the things it should and had an F1-score of 74.14 percent. Stochastic Gradient Descent did a lot better with these settings

than it did with a learning rate or a larger batch size. We got the results when we used Stochastic Gradient Descent with these specific settings.

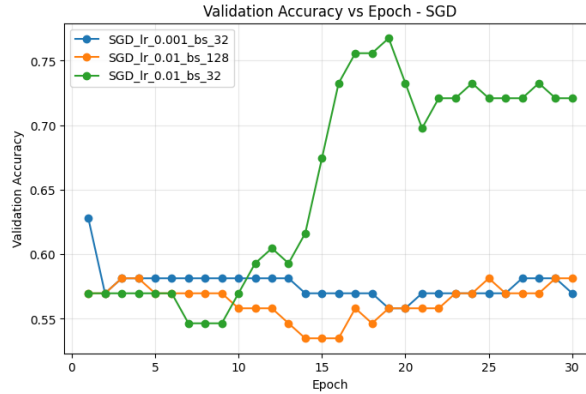


Fig. 7. Validation accuracy across epochs for the SGD configurations.

The main thing to take away is that Stochastic Gradient Descent can work well for this problem if you set it up correctly.. How well Stochastic Gradient Descent works really depends on how the learning rate and batch size and number of epochs all work together.

Stochastic Gradient Descent is good because it is simple and can be used on problems but you have to try a lot of different settings to get Stochastic Gradient Descent to give you good results.

15 Annex

The project repository includes the implementation, experimental results, and generated figures.

- GitHub implementation link: <https://github.com/rolandogonzlz>
- Dataset: Reduced malaria cell image dataset with two classes: Parasitized and Uninfected.
- Main notebook: `Celulas_Malaria_SGD_AoA.ipynb`
- Main generated outputs: summary metrics, convergence curves, confusion matrix, and new image prediction.

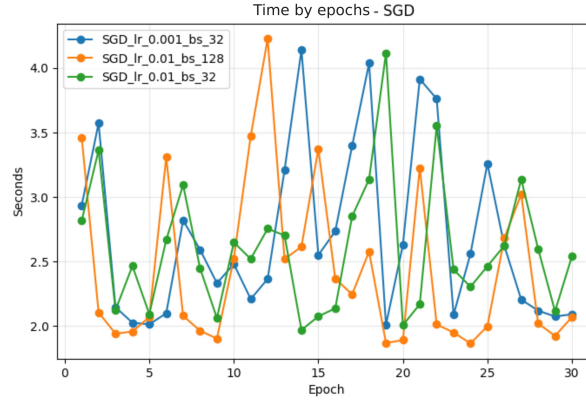


Fig. 8. Average epoch time behavior for the SGD configurations.

References

1. Robbins, H., Monro, S.: A stochastic approximation method. *The Annals of Mathematical Statistics* **22**(3), 400–407 (1951)
2. Bottou, L.: Large-scale machine learning with stochastic gradient descent. In: *Proceedings of COMPSTAT 2010*, pp. 177–186. Springer, Heidelberg (2010)
3. LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. *Nature* **521**, 436–444 (2015)
4. Goodfellow, I., Bengio, Y., Courville, A.: *Deep Learning*. MIT Press, Cambridge (2016)
5. Rajaraman, S., Antani, S.K., Poostchi, M., Silamut, K., Hossain, M.A., Maude, R.J., Jaeger, S., Thoma, G.R.: Pre-trained convolutional neural networks as feature extractors toward improved malaria parasite detection in thin blood smear images. *PeerJ* **6**, e4568 (2018)
6. TensorFlow: TensorFlow Keras Optimizers Documentation. https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/SGD, last accessed 2026/05/02